SYSTEM ARCHITECTURE / REPOSITORY SNAPSHOT

StockIt

Complete architecture

Indian equity research, machine learning and historical simulation.

26 October 2026 | Package v0.12.0

PHASE 12 SOFTWARE	Complete; validation passed
MARKET QUALIFICATION	Pending evidence and owner review
PHASE 13	Not authorized to begin

Inside this document

01-04 System shape, current status, modules and interfaces

05-08 Market data, qualification, time, features and datasets

09-11 Training, evaluation, registry, inference and simulation

12-14 HTTP services, dashboard, storage and engineering

15-16 Long-term roadmap, remaining work and phase gate

Implemented software and planned capabilities are identified separately. This architecture snapshot does not establish market validity, authorize trading or advance the project phase.

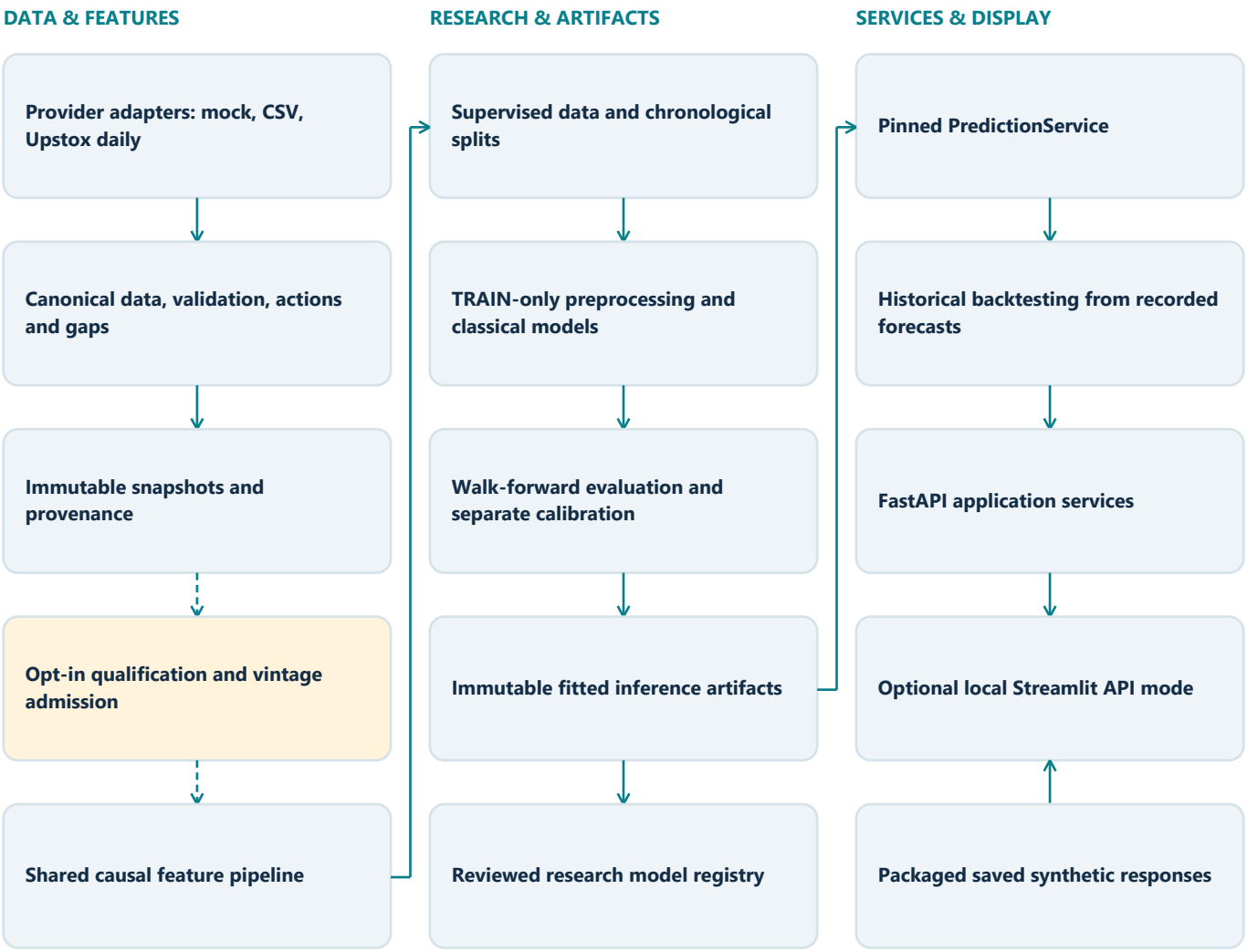
Repository-grounded snapshot: **26 October 2026 · package v0.12.0**.

StockIt is an independent Indian equity-market research and portfolio project. It has no relationship to BRIL Tech, Recverse, SeedNet or blockchain projects. Its initial ML task is next-trading-session direction classification, not exact price forecasting, guaranteed returns or live trade execution.

This document distinguishes implemented software, incomplete market qualification and planned extensions. It does not replace historical phase reports or advance the project gate. Earlier architecture documents describe their producing phases; the current source, newer dashboard guide and quote-parser follow-up are used here.

1. Architectural shape

StockIt is a **modular Python application** with explicit domain and repository interfaces. Training/research jobs, FastAPI and Streamlit may run as separate local processes, but reuse the same application/domain package. There is no implemented microservice mesh, distributed data platform or production cluster.



Snapshots also supply prediction and validated raw execution inputs. Features share their causal implementation with inference. Evaluation supplies recorded out-of-sample forecasts to backtesting; prediction supplies recorded responses. Prediction and backtesting both delegate through FastAPI. The packaged synthetic responses feed the dashboard directly.

The direct development-source path supports existing saved/synthetic workflows. It is not an automatic authorization path for real provider downloads. Phase 12 qualification is an explicitly composed admission layer, and the current provider-adjusted research path remains incomplete.

2. Current implementation and status

Capability	Actual state
Phases 1-11	Implemented software: foundation through model registry/Random Forest expansion
Phase 12 software	Implemented and validated; later offline quote-schema correction also validated
Upstox access	Bounded quote and historical probes returned HTTP 200 on 13 October; one actual Infosys historical observation validated
Real-market qualification	**PENDING**: representative pilot, reconciliation and other required evidence remain
Offline dashboard	Implemented separate workstream; synthetic saved content and optional existing local API
Daily OHLC quote parser	Implemented offline on 26 October; no new quote HTTP endpoint or real-data admission
Formal Phase 13	**NOT AUTHORIZED TO BEGIN**
Formal Phase 17 dashboard	Planned; the offline demo does not complete this phase
PostgreSQL, Docker production deployment, live brokers	Future work, not implemented operational infrastructure

The latest scoped quote correction observed 85 new tests and 267 affected tests passing, plus lint/format, builds and a wheel-import smoke check. Those are prior observed software checks, not new test runs for this architecture document and not evidence of market accuracy or profitability.

3. Repository and module arrangement

```
outputs/stockit/
├── pyproject.toml                  Package metadata and optional extras
├── uv.lock                         Locked dependency resolution
├── stockit.toml                    Non-secret application/storage defaults
├── src/stockit/
│   ├── core/                       Settings, paths, logging, application errors
│   ├── data/
│   │   ├── providers/              Provider interface, mock, local CSV, Upstox
│   │   │   └── upstox_quotes.py    Offline daily quote parser; no transport
│   │   ├── repositories/          Historical repository interface, Parquet/SQLite
│   │   ├── qualification/         Identity, session hours, vintages, quality,
│   │   │                          liquidity, protocol, opt-in admission
│   │   └── ...                    Canonical bars, validation, actions, gaps,
│   │                              hashes, snapshots and codecs
│   ├── features/
│   │   ├── groups/                Returns, trend, momentum, volatility,
│   │   │                          volume and price structure
│   │   ├── repositories/          Feature persistence
│   │   └── ...                    Schema, registry, causal pipeline and service
│   ├── supervised/
│   │   ├── repositories/          Supervised Parquet and SQLite catalog
│   │   └── ...                    Horizons, labels, eligibility, splits,
│   │                              dataset validation and service
│   ├── models/                    Preprocessing, Logistic Regression, Forest,
│   │                              training, diagnostics and artifact persistence
│   ├── evaluation/                Expanding walk-forward, metrics, calibration,
│   │                              reserved final-holdout protocol
│   ├── registry/                  Research candidates, comparisons and lifecycle
│   ├── prediction/                Pinned saved-model inference and provenance
│   ├── backtesting/               Strategy, simulation orders/fills, cash/shares,
│   │                              trades, costs, metrics and benchmark
│   ├── api/
│   │   ├── schemas/               Strict Pydantic HTTP envelopes
│   │   └── ...                    Routes, catalog, services, jobs and safe errors
│   ├── dashboard/                 Streamlit, Plotly, local API client and
│   │                              packaged synthetic demo JSON/hash
│   ├── cli.py                     Explicit workspace/configuration commands
│   └── __main__.py
├── config/calendars/               Bounded fixtures and reviewed calendar snapshot
├── tests/                         Layered offline/unit/integration/regression tests
├── examples/                      Explicit research/replay/example compositions
├── notebooks/                     Research only
├── docs/                          Architecture, policies, evidence and phase reports
├── .github/workflows/ci.yml        Lint/format/test/build workflow
└── var/                           Ignored local runtime storage
```

The surrounding workspace also contains separately delivered model examples, protected protocol reservations and diagnostic reports. Those are not automatically loaded or discovered by the API or packaged as a real-market dataset.

4. Layer responsibilities and interfaces

Layer	Main interfaces/objects	Responsibility and boundary
Foundation	Settings, WorkspacePaths	Explicit configuration and initialization; no market calculations
Ingestion	MarketDataProvider, MarketDataService	Normalize provider representations into StockIt-owned data; no ML logic
Canonical market data	SecurityIdentifier, MarketBar, MarketData	Security/date/OHLCV validation and explicit synthetic provenance
Historical processing	CorporateActionEngine, GapDetector, HistoricalDataset	Preserve observations, derive supported research views, audit actions/absences
Historical storage	MarketDataRepository	save/load/exists with explicit snapshot version/security/date scope
Qualification	ProviderCapture, InstrumentCatalog, SessionCalendar, CapturedVintage, VintageRepository, QualifiedPredictionSource	Source/evidence-aware capture and opt-in admission; not a provider accuracy certificate
Features	FeaturePipeline, FeatureService, FeatureFrame, FeatureRepository	Registered causal calculations, warm-up/reset reasons and exact schema
Supervised data	SupervisedDatasetBuilder, SupervisedService, SupervisedDataset, split policies	Labels, eligibility, horizons and both-endpoint chronological assignment
Training	train_baseline, TrainingRun, model configurations	Fit learned preprocessing/model only on designated TRAIN
Evaluation	EvaluationPlan, evaluate_walk_forward, calibration/holdout objects	Chronological OOS evaluation, separate calibration and protected test protocol
Artifacts	ModelArtifactRepository, InferenceArtifactRepository	Immutable fitted state, metadata, diagnostics and checksums
Registry	ModelRegistry	Reviewed research lifecycle and explicit model selection
Prediction	PredictionSource, RepositoryPredictionSource, PredictionService	Reconstruct causal features and run pinned frozen inference
Simulation	OOSBatch, BacktestConfig, run_backtest, BacktestResult	Consume recorded forecasts and simulate next-session execution
HTTP	ApiCatalog, ApiServices, JobManager	Resolve explicit resource/policy IDs; transport only
Presentation	Streamlit UI, ApiClient, saved demo loader	Display domain results; no duplicated analytics or trading

Provider logic stays in data adapters. UI and HTTP code never fit models or recalculate independent trading engines. Registry reads/promotions are not an authentication system. Importing modules must not start jobs or network requests.

5. Data ingestion, prices and qualification

Implemented inputs are an explicitly synthetic mock, a canonical local daily CSV provider, and a read-only Upstox V3 daily equity adapter. NSE/BSE identity syntax and calendar abstractions exist; actual tested connectivity was scoped to NSE. Do not infer full NSE/BSE/index coverage from adapter or symbol support.

The data path is:

```
Explicit provider + dated identity + requested completed sessions
    -> original payload / retrieval evidence where capture is used
    -> canonical StockIt records
    -> structural validation
    -> action/gap processing and immutable historical snapshot
    -> captured vintage and qualification report, when explicitly composed
```

Structural checks cover aware unique timestamps, one daily observation per IST session, positive finite OHLC, high/low relationships, integer nonnegative volume, security and requested scope. Missing observations are never fabricated. Gap classification separates weekends/holidays from missing observations, provider failure, unavailable symbols and action discontinuities.

Historical snapshots distinguish raw returned OHLCV from derived research values. Policy 1.0 applies supported split/bonus research adjustment only to proven unadjusted input with complete knowable action evidence. Unsupported dividends, rights, mergers/demergers and ambiguous actions remain flagged/excluded.

Upstox-specific current limitation: its reply states prices are back-adjusted for splits/bonuses, dividends are not adjusted, and old candles can be revised. Original provider bytes are not necessarily economically unadjusted prices. The adapter still defaults to UNKNOWN basis and existing real admission rejects provider-adjusted/unknown data. A versioned supported research path remains pending; never double-adjust these values or silently use them as raw execution prices. Historical volume adjustment and unadjusted execution-price availability still need evidence.

The offline quote parser is a separate boundary: daily prev_ohlc=null and zero index volume are accepted. Zero equity quote volume stays ZERO_UNVERIFIED. A structurally valid quote is not a completed historical candle, tradable liquidity, or a qualified research source. No live quote transport/API route was added.

6. Calendar, time and revision architecture

There are distinct concepts:

- Provider candle label / interval start.
- Canonical completed-session anchor.
- Provider publication time, when supported for that exact revision.
- Actual retrieval and ingestion time.
- Feature decision cutoff and simulated signal/execution time.

Operational instants use aware UTC; Indian session dates and display use Asia/Kolkata and INR. Calendar coverage is explicit and bounded; special sessions override ordinary weekday assumptions. The checked-in reviewed NSE 2026 snapshot has unresolved special hours; qualified BSE/2027 coverage is not supplied in the current checkout. This is a repository observation, not a claim that no newer exchange notices exist externally.

Availability is conservative: the maximum of event completion, supplied supported publication, retrieval and ingestion. Upstox's next-morning statement does not provide a universal clock/SLA or prove availability before the next open. Current downloads cannot recreate the revisions available at older decisions. New captures create immutable vintages; comparison records revisions without overwriting old ones.

The fresh 4 January-31 March 2027 holdout remains reserved and unconsumed. A reservation is not validated market data or an approved evaluation outcome.

7. Feature engineering

The implemented schema contains **46 registered features in six groups**:

Group	Features
Returns (6)	return_1d, return_5d, return_10d, return_20d, log_return_1d, log_return_5d
Trend (12)	sma_5/20/50/200, ema_5/20/50, price_vs_sma20/50/200, sma20_vs_sma50, sma50_vs_sma200
Momentum (9)	rsi_14, macd, macd_signal, macd_histogram, roc_5/10/20, momentum_5/10
Volatility (11)	atr_14, rolling_volatility_5/10/20, high_low_range, open_close_range, bollinger_middle/upper/lower/width/position
Volume (4)	volume_change_1d, volume_sma_20, volume_ratio_20, volume_zscore_20
Price structure (4)	intraday_return, high_low_range_pct, close_position_in_range, gap_pct

The current Phase 4 input policy is **unadjusted raw OHLCV with causal segment resets** around gaps/actions, rather than arbitrary provider-adjusted candles. That distinction matters to the pending Upstox research integration. Calculators share a registry/schema, use backward-looking observations only and record warm-up and undefined-value reasons. Missing feature values are not hidden future fills. Future-data mutation and prefix reconstruction are part of validation.

8. Supervised datasets

For an admitted anchor t and its next expected trading session t+1:

```
next_session_return = research_close[t+1] / research_close[t] - 1
next_session_direction = 1 when return > 0; otherwise 0
```

Zero return belongs to class 0. Features and targets are separate exports. Each sample records security, feature/label timestamps, expected session, horizon, eligibility, exclusions, gap/action evidence and split assignment. Warm-up, invalid source/price, missing session and unsupported-action rows remain in the audit dataset rather than silently disappearing.

Both feature and label dates must lie in the same split. For example, a TRAIN anchor on its last date with a label in VALIDATION is excluded from TRAIN. The splitter does not randomly shuffle data or choose windows from their outcomes.

Parquet stores samples.parquet plus train.parquet, validation.parquet and test.parquet under a versioned dataset directory. Optional SQLite catalogs retain metadata. Source revisions require recomputed features; later labels cannot silently replace the feature source's historical observations.

9. Training, evaluation, calibration and artifacts

Implemented models are the Logistic Regression baseline and Random Forest, with a TRAIN-derived majority comparator. XGBoost is deferred; neural models are not implemented. PyTorch is not a current dependency.

Training uses a saved ordered predictor manifest, guard, optional configured imputation and shared scaling/model pipeline. Statistics are fitted only on the selected TRAIN interval. Empty observed training columns, schema mismatches and incompatible fitted state fail explicitly. Configuration, seeds and dependency versions are retained, without promising cross-platform bitwise repeatability.

Evaluation uses expanding chronological windows, fresh fits per fold, disjoint validation windows and horizon-safe sample assignment. Rolling walk-forward is not implemented. Reports include classification/probability metrics, confusion counts, majority comparisons and fold variation. Undefined metrics retain their reasons; score improvements do not automatically choose or deploy a winner.

Calibration has separate ordered roles:

```
Final base TRAIN -> CALIBRATION_FIT -> CALIBRATION_EVALUATION -> FINAL_HOLDOUT
```

A frozen estimator prevents refitting the base pipeline during sigmoid calibration. The calibrator is assessed on a later disjoint interval. Final holdout access is governed by the reserved protocol and is not used for tuning. No fresh real-market holdout scoring is authorized by this document.

Model bundles are immutable and content-addressed:

```
models/<model_version>/
  model.joblib
  metadata.json
  diagnostics.json
  checksums.json
```

They bind preprocessing, model/calibrator, predictor order and numerical schema, dataset/feature/evaluation identities, policies, software/dependencies and complete inherited provenance. Canonical model-state identity and serialized byte hash are different integrity concepts. Loading requires explicit operator trust and compatibility checks; checksums do not make arbitrary joblib files safe.

10. Registry and prediction

The SQLite research registry stores model metadata and reviewed decisions, not a second independent model store. Lifecycle states include CANDIDATE, VALIDATED, CHAMPION, RETIRED and ROLLED_BACK. Promotion requires explicit review and current candidate/champion revisions. Transactions and a unique-champion constraint protect concurrent switches; rollback records both sides and displaced models require fresh validation before ordinary promotion.

Audit hashes detect corruption but are not digital signatures or access control. A champion is a reviewed research selection, not a market-validity certificate. The API uses an explicit catalog snapshot; changing the registry does not hot-swap an already configured API model. No public promotion/rollback endpoint exists.

PredictionService receives an explicit source, calendar, model artifact and cutoff/staleness policies. It validates the full chosen snapshot, reconstructs the same causal feature recipe, verifies schema/runtime/fitted state and performs frozen inference. It does not fit, tune, recalibrate or alter thresholds.

The response records direction, probability_up/down, threshold, feature timestamp, next session, model/calibration/schema versions, as-of and complete provenance. Probability_up always means P(UP), even when the displayed decision is DOWN; it is not expected profit. The request as-of is not wall-clock generation time.

QualifiedPredictionSource adds opt-in vintage/universe admission. Existing saved RepositoryPredictionSource workflows remain available; the default HTTP service does not automatically fetch, qualify or promote newly downloaded real data.

11. Historical backtesting

The simulation flow is:

```
Recorded OOS forecast -> strategy intent -> simulated order
    -> next-session OPEN fill with fees/slippage
    -> cash/shares -> closed trades/open position
    -> session-close equity -> metrics and same-period benchmark
```

Inputs are sealed walk-forward forecasts or verified recorded inference responses, plus matching evidence, raw execution prices, calendar, policies and protected periods. The engine never generates its own predictions or trains a model.

V1 is single-security, long-only, cash-funded, whole-share simulation. It has explicit missing-signal behavior, no borrowing/shorting and no forced final exit; open positions are marked at raw close. Current gap/action guards reject affected evaluation periods rather than inventing fills or implementing full entitlement accounting. It is not a live portfolio or broker ledger.

Outputs include orders, executions, decisions, trades, equity, final portfolio, total return, CAGR, Sharpe, drawdown, win rate, trade count, benchmark and versioned provenance. Costs/slippage are configured assumptions, not a verified complete Indian tax/fee model. Daily bars cannot establish queues, partial fills or impact.

12. API and job architecture

FastAPI owns strict HTTP validation, resource binding, pagination, safe errors, correlation IDs and application-scoped dependencies. ApiServices delegates domain work. The default catalog is empty, artifact trust is false, and startup performs no provider retrieval or model fitting. Server-controlled resource/policy/model IDs replace arbitrary client filesystem paths or implicit latest selection.

Method	Existing endpoint
GET	/v1/health
GET	/v1/stocks
GET	/v1/stocks/{symbol}
GET	/v1/stocks/{symbol}/history
GET	/v1/stocks/{symbol}/indicators
GET	/v1/models
GET	/v1/model-performance
POST	/v1/predict
POST	/v1/backtests
GET	/v1/backtests/{job_id}
GET	/v1/backtests/{job_id}/result

The API exposes saved resources. History needs a pinned source/security; indicators read persisted features rather than independently computing them. Prediction and backtest requests pin original source times, model/schema/calibration identities, calendars and policy selections. Server-owned reservations cannot be changed by client echoes. Performance reads a sealed development report, never final holdout.

Backtests can return synchronous results or HTTP 202 jobs. The bounded local JobManager has queued/running/completed/failed states and result references. Jobs are process-local and non-durable, not a distributed queue. API liveness is not market-data readiness. OpenAPI/docs/redoc describe the transport contract.

This local API has no implemented production authentication/authorization, TLS termination, tenancy or distributed durability. Typed failures are mapped to safe HTTP envelopes; logs use correlation and resource references, not credentials.

13. Dashboard and developer experience

The optional Streamlit/Plotly demonstration has five pages: Overview, Predictions, Backtests, Model Registry, and Data & Qualification. Default saved mode loads only packaged synthetic JSON plus its hash and needs no token, internet, model file or running API after dependencies are installed.

Local API mode uses a configurable loopback client with schema validation, timeouts, safe errors, proxy bypass, no redirects and bounded job polling. POSTs require explicit user buttons; pending jobs can be resumed, terminal jobs stop. There is no second prediction or backtest engine in the UI.

The supplied local demo server registers the saved Logistic Regression profile. Random Forest predictions are saved offline views. Registry lifecycle and artifact verification are dated export evidence, not a live champion selector. The UI has no lifecycle mutations, broker credential fields or trading controls. It remains synthetic even though a separate provider diagnostic succeeded.

This workstream does not complete formal Phase 17, whose broader application dashboard remains planned.

14. Storage, provenance, configuration and testing

WorkspacePaths defaults to the ignored var/ root:

```
var/
  data/raw/
  data/processed/
  data/features/
  data/supervised/
  models/
  reports/
  metadata.sqlite3
```

Repository constructors receive explicit backend paths; additional registry and vintage catalogs are configured at composition time rather than universally invented filenames. Parquet is analytical storage, SQLite holds local records/ catalogs/registry evidence, JSON carries manifests/reports and joblib holds trusted fitted artifacts. PostgreSQL is a future repository adapter, not a running backend.

SHA-256 versions bind original data, retrieval evidence, transformations, action/ calendar/gap decisions, feature schema/recipe, labels/splits, model/calibration/ evaluation state and execution policies. Original producer versions survive newer software reads. Local publication uses staging and atomic operations where supported; idempotence does not mean overwriting corrupt or revised data.

Settings precedence is defaults < TOML < actual environment. StockIt does not silently load .env. Storage initialization is explicit. Secrets remain outside tracked source and diagnostic outputs. Core has no dependency on models/UI.

The stack is Python 3.11+, pandas/NumPy/PyArrow, scikit-learn/joblib, FastAPI/ Pydantic/Uvicorn, optional Streamlit/Plotly, pytest/Ruff/build and uv locking. Tests cover structural math, causality, future mutation, horizon/split boundaries, storage failure/integrity, calibration, frozen inference, registry concurrency/ rollback, simulation, HTTP errors/reservations and offline dashboard behavior. CI is configured for Python 3.11/3.12 lint, format, tests and distribution builds; this document does not claim a fresh CI execution.

15. Long-term architecture and phase boundaries

These are planned capabilities, not current executable modules:

Phase	Planned responsibility
13	Advanced features, signals and market regimes; requires explicit authorization
14	Risk and portfolio controls, distinct from prediction and historical simulation accounting
15	Paper trading and virtual execution before any live broker use
16	Monitoring and drift detection; no automatic retraining from one bad prediction
17	Full dashboard over application/API services
18	Production infrastructure/security: deployment, Docker, authentication, operational controls and durability
19	Separately controlled broker integration
20	Full production validation
21	Controlled continuous model lifecycle

XGBoost and any neural experiments remain deferred and require justified dependencies and the same temporal evaluation discipline. Scaling may replace storage/job implementations behind interfaces; it does not require prematurely splitting every domain into a microservice.

The future decision chain must keep model probabilities, signals, risk approval, portfolio decisions and execution separate. Backtest order records are not broker orders. No existing HTTP handler places, modifies or cancels real orders.

16. Immediate architectural work remaining

1. Complete an evidence-bound provider-adjusted research policy without double adjustments, and resolve volume/unadjusted execution-input requirements.
2. Freeze a bounded NSE pilot with dated identity/calendar/action scope and an authorized independent reference before inspecting outcomes.
3. Capture actual immutable vintages; measure coverage, gaps, revisions and reference agreement; document universe and execution limitations.
4. Freeze the real evaluation protocol while protecting the future holdout.
5. Produce the evidence report for explicit owner review and transition approval.

The market-qualification gate remains mandatory. Software checks, a provider reply, a reserved holdout or a successful connection alone cannot authorize advancement.

PHASE 12 - SOFTWARE IMPLEMENTATION: COMPLETE; VALIDATION PASSED

PHASE 12 - MARKET QUALIFICATION: PENDING

PHASE 13 - NOT AUTHORIZED TO BEGIN

Repository references

- Source package
- Earlier architecture decisions
- Controlled roadmap
- Market gate
- Feature schema
- Storage
- Vintages
- Provider correspondence
- Daily quote parser
- Supervised data
- Calibration
- Registry
- Inference artifacts
- Backtesting
- HTTP services
- Offline dashboard
- Latest scoped quote-fix report

Created from the current source and phase documentation. No runtime files were changed, API requests performed, credentials accessed or tests rerun for this architecture-only task.